



Universidad de Jaén

Tema 4.3 Programación en el Cliente: Intercambio de datos con el servidor

Desarrollo de Aplicaciones Web

José Ramón Balsas Almagro
Departamento de Informática
Universidad de Jaén
jrbalsas@ujaen.es

Este obra está bajo una [Licencia Creative Commons](http://creativecommons.org/licenses/by/4.0/)
Atribución-CompartirIgual 4.0 Internacional.

v 1.4.4

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Objetivos

- Conocer las características y **arquitectura** de una aplicación web centrada en el cliente
- Conocer los fundamentos de la **notación JSON** como método de intercambio de información con el servidor
- Conocer los fundamentos de los **servicios web** en general y **REST** en particular
- Saber implementar un servicio web CRUD básico en JEE con **JAX-RS**
- Conocer el concepto de **petición asíncrona AJAX** y saber cómo gestionarlas desde Javascript en el navegador



Aplicaciones web centradas en el cliente

● Motivación

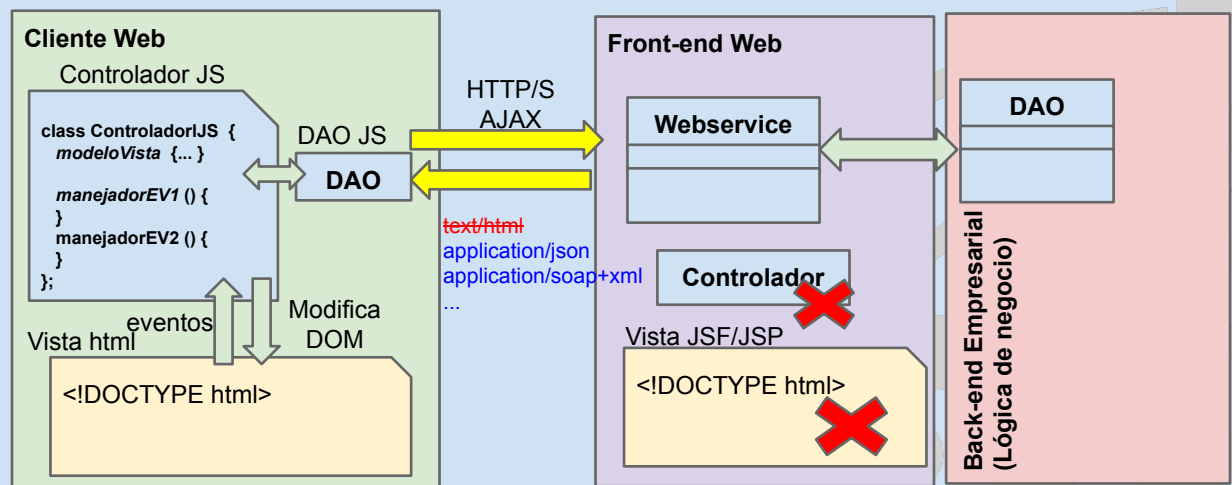
- El avance de las especificaciones HTML5, CSS y Javascript
- Compatibilidad y potencia de los navegadores actuales
- La WWW como plataforma ¿universal?

● Características

- Lógica de control en el cliente
- Arquitecturas MVC
- Interacción asíncrona (AJAX) con servidor
- Intercambio de *datos* con servidor en formatos estructurados: XML, JSON, ...
- Aplicaciones con Interfaz Enriquecida (RIA)
- Acceso a recursos locales: geolocalización, almacenamiento local, base de datos local, procesamiento en segundo plano, cámara web, ...

3

Arquitectura de aplicación web centrada en cliente



4

Procesamiento JSON en aplicaciones web

Introducción a los servicios web

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Formatos estructurados de información en Apps Web

● Motivación

- Facilitar el intercambio de información entre *cualquier* cliente y servidor
- Diferencias en la representación de tipos de datos entre lenguajes. e.g. objetos en java y javascript
- HTTP utiliza texto para la transferencia de datos

● Tipos

- **XML** (eXtensible Markup Language)
 - Meta-lenguaje basado en etiquetas y atributos
 - Permite verificación formal
- **JSON** (*Javascript Object Notation*)
 - Representación de objetos literales simples de Javascript
 - Simplicidad de representación y procesamiento

- Existen bibliotecas para su procesamiento en la mayoría de lenguajes de programación



Serialización de objetos

● **JavaScript Object Notation (JSON)**

- Representación de objetos literales en Javascript:
 - **sintaxis de arrays y arrays asociativos**
 - "propiedades" entrecomilladas
- Independiente del lenguaje
- Bibliotecas en la mayoría de lenguajes
 - En Jakarta: *JSON Processing and Binding APIs*
- Utilizado para la transferencia, recepción y almacenamiento de **información estructurada** en formato texto
- Sólo soporta atributos, NO métodos;

```
{
  "Id": 23,
  "Nombre": "Paco",
  "Edad": 28,
  "Socio": true,
  "servicios": ["tenis", "natación"]
}
```

● **Objeto nativo JSON en Javascript. Métodos:**

- **JSON.stringify(object)** devuelve la representación literal de un objeto. e.g. para enviarla por http
- **JSON.parse(cadena)** crea un objeto a partir de su notación literal. e.g. para recibir un objeto por http

7

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Serialización de Objetos en Javascript

Ejemplo:

```
let persona = {
  nombre: "Juan",
  apellidos: "García López",
  edad: 23,
  tfns: [953223344, 567888444, 666777373],
  empleos: [ {tipo:"Cartero",sueldo:1200} , {tipo:"Pintor",sueldo:900} ]
};

persona.nombre;           //Juan
persona.tfns[0];           //953223344
persona.empleos[1].sueldo; //900

let cadena = JSON.stringify(persona); //convierte el objeto a una cadena
let mellizo = JSON.parse(cadena);     //Nuevo objeto
mellizo.apellidos;                     //García López

let trillizo = JSON.parse(JSON.stringify(persona)); //copia de objetos! (solo atrib.)
```

8

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Serialización de objetos en JEE

- La especificación JakartaEE dispone de diferentes módulos para la serialización de objetos:
 - **JAXB**, *Jakarta XML Binding* v.4, Permite la transformación de objetos java en XML y viceversa.
 - **JSON-P**, *JSON Processing* v.2.1, manipulación de contenido JSON
 - **JSON-B**, *JSON Binding* v.3. Transformación entre objetos java y JSON

```
Cliente cliente = new Cliente("Pepe","12345678-Q", true);
```

```
// Create Jsonb and serialize
```

```
Jsonb jsonb = JsonbBuilder.create();
```

```
String result = jsonb.toJson(cliente); //result= {"nombre":"Pepe","dni":"12345678-Q","socio":true}"
```

```
// Deserialize back
```

```
Cliente c = jsonb.fromJson("{\"nombre\":\"Pepe\",\"dni\":\"12345678-Q\",\"socio\":false}", Cliente.class);
```

9

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Introducción a los servicios web

- "A web service is a software system designed to support interoperable machine-to-machine interaction over a network."

— W3C, Web Services Glossary

- Diferentes tipos de clientes: web, móvil, escritorio, IoT...
- También conocidos como APIs

- **Protocolos de comunicación**

- **SOAP**, *Simple Object Access Protocol*, basado en XML
- **REST**, *REpresentational State Transfer*, basado en métodos HTTP, sin estado, para operaciones CRUD y JSON para intercambio estructurado
 - Create: **POST** [/api/clientes](#)
 - Read: **GET** [/api/clientes/1](#)
 - Update: **PUT** [/api/clientes/1](#)
 - Delete: **DELETE** [/api/clientes/1](#)

10 ○

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Servicios web en JakartaEE

● Características principales

- Enrutamiento URLs a métodos
- Transformación transparente de objetos

● Módulos JakartaEE

- **Jakarta XML Web Services, JAX-WS, v 4.0**
 - Servicios web "grandes" basados en SOAP y XML
- **Jakarta RESTful Web Services, JAX-RS, v 3.1**
 - Servicios web "ligeros" basados en REST
 - Implementación de referencia, Jersey



11

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Creación de un servicio JAX-RS

Código de referencia

<https://github.com/jrbalsas/dawClubAJAX>

// JAXRS declarative endpoint registration and routing

```
import javax.ws.rs.ApplicationPath;
import javax.ws.rs.core.Application;
```

```
@ApplicationPath("api")
```

```
// Service URL: /api/*
```

```
public class ClubRESTService extends Application {
}
```

dawClubAJAX.git	
clientes [GET]	ClientesJSONService
HTTP Server JEE RESTful WS Server	
clientes [POST]	ClientesJSONService
HTTP Server JEE RESTful WS Server	
clientes/{id} [GET]	ClientesJSONService
HTTP Server JEE RESTful WS Server	
clientes/{id} [PUT]	ClientesJSONService
HTTP Server JEE RESTful WS Server	
clientes/{id} [DELETE]	ClientesJSONService
HTTP Server JEE RESTful WS Server	

com.daw.club.webservice.ClientesJSONService

```
@Path("clientes") //URLs: /api/clientes/...
```

```
@Produces("application/json")
```

```
@RequestScoped
```

```
public class ClientesRESTService {
```

```
@GET
```

```
public List<Cliente> getClientes() { ... }
```

```
@GET @Path("/{id}")
```

```
public Response getCliente( @PathParam("id") Integer id { ... }
```

```
@DELETE @Path("/{id}")
```

```
public Response borraCliente( @PathParam("id") Integer id { ... }
```

```
@POST @Consumes(MediaType.APPLICATION_JSON)
```

```
public Response creaCliente( @Valid Cliente c ) { }
```

```
@PUT @Path("/{id}") @Consumes(MediaType.APPLICATION_JSON)
```

```
public Response modificaCliente( @Valid Cliente c,
                                @PathParam("id") Integer id ) { ... }
```

```
}
```

12

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Anotaciones servicios JAX-RS

Anotación	Descripción
<code>@Path("/uri")</code> <code>@Path("/{param1}/...")</code>	Clase: ruta relativa del servicio web asociada a la clase Método: ruta relativa del servicio asociada al método. Admite parámetros en PathInfo, e.g. <code>@Path("/{id}")</code>
<code>@RequestScoped</code>	Persistencia del objeto que atiende el servicio. Por definición de servicio REST debe existir solo en ámbito de petición (Sin estado)
<code>@Consumes("mime")</code> <code>@Produces("mime")</code>	Asociado a clase (por defecto) o a métodos específicos. Tipos de datos que admite o genera el método/métodos del servicio, e.g. <code>text/html</code> , <code>application/json</code> , <code>application/xml</code> , <code>application/x-www-form-urlencoded</code> (POST formularios) o Constantes del tipo <code>MediaType.APPLICATION_JSON</code>
<code>@GET</code> , <code>@POST</code> , <code>@PUT</code> , <code>@DELETE</code> , ...	Asociado a métodos, indica el tipo de petición que atenderá el método
<code>@PathParam("param")</code> <code>@QueryParam("param")</code> <code>@FormParam("param")</code> <code>@CookieParam("param")</code> <code>@HeaderParam("param")</code>	Asociado a parámetros de métodos, indica el origen del parámetro (en Path, QueryString, en envío POST,...) que se convertirá y pasará (inyección) al parámetro. Pueden ir acompañados de <code>@DefaultValue("valor_defecto")</code> si no aparecen

13

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Envío de datos desde servicio JAX-RS

- Los datos devueltos por los métodos del servicio se convierten al tipo mime indicado por la anotación `@Produces`. Si método es void, status **204 (no content)**
- Los **tipos de datos estructurados**, objetos y arrays, pueden convertirse automáticamente (módulo JSON-B) a representaciones JSON usando el tipo MIME `application/json`
- Si se quieren controlar más detalles de la respuesta, e.g. código de estado, o añadir información adicional, puede utilizarse el tipo **Response (patrón builder)**

```
@Path("clientes")
@Produces(MediaType.APPLICATION_JSON)
@RequestScoped
public class ClientesRESTRService {
```

```
    @Inject @DAOMap
    ClienteDAO clienteDAO;
```

```
//Opción 1: sin control de código de estado
```

```
@GET
public List<Cliente> getClientes() {
    //status 200 and [ {"id":1,"nombre":"..."}, {...}, ...]
    return clienteDAO.buscaTodos();
}
```

```
//Opción 2: control de código de estado
```

```
@GET
public Response getClientes() {
    List<Cliente> clientes = clienteDAO.buscaTodos();
    Response response;
    response=Response.status( Response.Status.ACCEPT )
        .entity( clientes ).build();
    return response;
}
```

14 }

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Recuperación de datos en servicio JAX-RS

- Los **datos deben ser enviados en el tipo MIME** (encabezado content-type) que acepte el servicio o método con anotación `@Consumes(...)`. e.g
 - `text/html`
 - `application/x-www-form-urlencoded` para envíos de formularios
 - `application/json`, para objetos/arrays recibidos en JSON
- Los **datos simples**, *String*, *Integer*, *int*, *Float*, ... pueden convertirse automáticamente al tipo del parámetro asociado según las anotaciones.

```
@GET
@Path("/{id}")
public Response getCiente(@PathParam("id") Integer id) { --- }
```

- Los **datos estructurados**, e.g. *application/json*, en peticiones POST/PUT pueden convertirse y validarse automáticamente (módulo JSON-B) a *Beans* con propiedades equivalentes

```
@POST
@Consumes(MediaType.APPLICATION_JSON)
public Response creaCiente( Cliente c ) { }
```

15

Ejemplo de recuperación de datos

//Ejemplo de petición: GET /api/clientes/1

```
@GET
@Path("/{id}")
public Response getCiente(@PathParam("id") int id) {
    Response response;
    Cliente c = clienteDAO.buscald(id);
    if (c!=null) { //return status 200 and JSON {"id": 1, "nombre":"..."}
        response= Response.ok(c).build();
    } else {
        //Generate error messages as Arrays of Objects
        List<Map<String,Object>> errores = new ArrayList<>();
        Map<String,Object> err = new HashMap<>(); //temp DTO
        err.put("message", "El cliente no existe");
        errores.add(err); //Add additional errors if needed
        //Error 400 and JSON [{"message":"El cliente no existe"}]
        response=Response.status(Response.Status.BAD_REQUEST)
            .entity(errores).build();
    }
    return response;
}
```

- Una lista java se transformará en un vector json
- Un *map<string,Object>* se transforma en un objeto json con tantas propiedades como claves tenga
- Con java +14 se pueden usar *Records* para crear DTOs

- Ejemplo:

```
[ {key1: value1, key2: value2}, ... ]
```

//Ejemplo de petición: DELETE: /api/clientes/1

```
@DELETE @Path("/{id}")
public Response borraCiente(@PathParam("id") int id) {
    Response response;
    if (clienteDAO.borra(id)==true) {
        response= Response.ok(id).build();
    } else { //Error messages
        //...idem, customer does not exist
    }
    return response;
}
```

16

Ejemplos de métodos JAX-RS

```
// POST (create) /api/cliente
@POST @Consumes(MediaType.APPLICATION_JSON)
public Response creaCliente(@Valid Cliente c) {
    Response response;
    if ( clienteDAO.crea(c)==true ) {
        Integer newId=c.getId();
        response= Response.ok(c).build();
    } else { //Error messages
        List<Map<String,Object>> errores=new ArrayList<>();
        Map<String,Object> err=new HashMap<>();
        err.put("message", "No se ha podido crear el cliente");
        err.put("cliente", c);
        errores.add(err);
        response=Response.status(
            Response.Status.BAD_REQUEST)
            .entity(errores).build();
    }
    return response;
}
```

```
// PUT (update) /api/cliente/1
@PUT @Path("/{id}")
@Consumes(MediaType.APPLICATION_JSON)
public Response modificaCliente( @Valid Cliente c,
    @PathParam("id") Integer id) {
    Response response;
    c.setId(id);
    if ( clienteDAO.guarda(c) ) {
        response= Response.ok(c).build();
    } else { //Error messages
        //Idem, customer can not be updated
    }
    return response;
}
```

17

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Validación de datos

- JAX-RS permite interactuar con **Bean-Validation**
- **Tipos:**
 - Utilizando **anotaciones Bean-Validation** junto a parámetros


```
@POST
@Path("/crea")
@Consumes( MediaType.APPLICATION_FORM_URLENCODED )
public void crearCliente (@Size(min=4,max=25) @FormParam("name") String nombre,
    @Pattern(regexp="\d{7,8}(-?[a-zA-Z])?" @FormParam("dni") String dni,
    @NotNull @FormParam("socio") Bool socio) { ... }
```
 - Anotación **@Valid** para validación de beans


```
@POST
@Consumes(MediaType.APPLICATION_JSON)
public Response creaCliente (@Valid Cliente c) { ... }
```
 - Si hay errores, se genera **excepción ConstraintValidationException**
 - El método **NO** se ejecuta
 - Por defecto, código de estado devuelto: **400 (Bad Request)**
 - Se puede definir un **manejador de excepción:**
 - La excepción lleva asociado el conjunto de violaciones de reglas de validación: **e.getConstraintViolations()**
 - Se puede usar el manejador para generar el **contenido JSON con información de los errores** y enviar al cliente

18

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Manejador de excepción *ValidationException*



@Provider

```
public class ClubValidationExceptionHandler  
    implements ExceptionMapper<ConstraintViolationException> {
```

@Override

```
public Response toResponse(ConstraintViolationException e) {  
    //convert bean-validation constraintviolation set to json array
```

```
List< ErrorValidacion > errors = new ArrayList<>();
```

```
for (ConstraintViolation<?> cv : e.getConstraintViolations()) {
```

```
    //attribute name is the last part, e.g. method.arg0.propname  
    String[] parts=cv.getPropertyPath().toString().split("\\.");
```

```
    errors.add( new ErrorValidacion(parts[parts.length-1], cv.getMessage() ) ); //temp DTO for bean validation error
```

```
public record ErrorValidacion (String name,  
                               String message ) {}
```

Contenido JSON devuelto

```
[ {"name":"nombre" , "message":"El formato.. "}, {...} ]
```

El objetivo del manejador es enviar información al cliente en formato JSON sobre los errores que se han producido en la validación de los datos suministrados

Creamos objetos temporales con Java Records (jdk 14+) para contener los mensajes de error:

```
};  
return Response  
    .status(Response.Status.BAD_REQUEST)  
    .entity(errors)  
    .build();  
}
```

19

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Peticiones asíncronas al servidor

AJAX



Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Conexiones HTTP desde javascript

- Tradicionalmente javascript podía iniciar la **conexión http** de dos formas:
 - Cambiando la propiedad `window.location`
 - Llamando al método `form.submit()`
- La página debe ser *recargada totalmente* del servidor → lentitud
- En 2000 se crea en IE5 la primera API para realizar conexiones asíncronas al servidor para intercambiar información parcial
- Estandarizado mediante el objeto **XMLHttpRequest** (IE7+)
 - (IE<7) `ActiveXObject("Microsoft.XMLHTTP")`
 - Sintaxis compleja
- Nueva función **fetch**, especificación WHATWG,
 - Chrome, 2016. Edge y Safari, 2017. Firefox, 2018. No IE
 - Más fácil de usar

21

Conexiones HTTP asíncronas

- **AJAX, Asynchronous Javascript XML.**
 - *Asíncronas: el navegador no sustituye la página con los datos descargados*
 - Datos de diferentes tipos: json, texto, xml, jpeg, mp3, etc.
 - Por defecto, sólo hacia el servidor de la página, *Origin*
 - Para otros destinos, el servidor destino debe autorizar el tipo de petición desde el sitio origen: *Cross-Origin Resource Sharing* o **CORS**
- **Tipos**
 - **Iniciadas por cliente usando fetch o XHR** ← **nos centraremos en estas**
 - **Server-Side Events**
 - API JS **EventSource** en cliente para recepción de mensajes desde el servidor
 - El código JS establece una conexión con el servidor que el navegador mantiene actualizada
 - El código JS define manejadores que se ejecutan en respuestas a envíos del servidor
 - Soportada en Jakarta REST Web API, JAX-RS
 - **WebSockets**
 - Protocolo bidireccional sobre HTTP
 - Soportado por la mayoría de navegadores actuales
 - Soportada en especificación Jakarta websockets

22

Conexión HTTP asíncrona

- Se utiliza un objeto o llamada por conexión

`fetch(...)`

y en jQuery... `$.ajax(...)`

o tradicionalmente

`new XMLHttpRequest()`

- Elementos de una petición HTTP asíncrona

- Las habituales: método (GET/POST), url, atributos de cabecera opcionales y cuerpo en caso de mensajes POST, etc.

- Respuesta HTTP

- Código de estado para conocer el éxito de la conexión
- Encabezados HTTP. e.g. *Content-Type: application/json*
- Cuerpo de la respuesta, e.g. `{ "propiedad1": valor1, ... }`

23

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Peticiones asíncronas con fetch

- `fetch ( url [, { Request_config } ] )`

- Devuelve un **objeto Promise** para evitar el anidamiento de llamadas asíncronas: *callbacks*

- Representa el resultado futuro (o fallo) de una llamada asíncrona.

```
let op= new Promise ( ( resolv_func, reject_func ) => {           let op= new Promise ( func_async )
    /* Operación asíncrona */                                   .then ( resolv_func [, reject_func ] )
    })                                                         .catch ( reject_func )
```

- Se ejecuta *resolv_func* o *reject_func* dependiendo del éxito o fallo

- Reciben los datos generados por la operación asíncrona.

- Si un manejador devuelve una nueva promesa se pueden enlazar

- El **manejador de fetch** (*resolv_func*) recibe un objeto **Response**.

- Objeto Response** [+info](#)

- Propiedades para consultar el estado de la petición: *status*, *ok*, *statusText*...

- Métodos con datos (promesas!) de respuesta: *.text()*, *.json()*, ...

```
fetch( url ).then( response => {                                //Empieza la respuesta, pero todavía puede que no se hayan recibido todos los datos
    console.log(response.headers.get("Content-Type"));
    console.log(response.status);
    console.log(response.statusText);
    console.log(response.url);
    return response.text();                                     //Devolvemos promise para recuperar datos de respuesta... cuando estén disponibles
}).then ( texto => console.log(texto) );                         //Encadenamiento de Promise: ya tenemos los datos
```

24

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Ejemplo petición asíncrona con fetch

2

```

window.onload = () => {
  document.getElementById( "btJson" ).onclick = () => {
    fetch( 'https://rest_service_url' )
      .then( response => response.json() )
      .then( datos => verDatos(datos) )
      .catch( err => {
        console.log( `Error de conexión ${err.message}` );
      });
  };
};

function verDatos( obj ) {
  let dv1 = document.getElementById("id1");
  var txt = "";
  for (let prop in obj) {
    txt += `<p><strong>${prop}</strong>:${obj[prop]}</p>`;
  }
  dv1.innerHTML = txt;
};

```

1

```

<body>
  <div id="id1">Ejemplo JSON</div>
  <button id="btJson" >Descarga datos</button>
</body>

```

3 respuesta json de https://rest_service_url

```
{ "name": "Pepe", "Apellidos": "García Sánchez", "age": 34 }
```

4

```

name:Pepe
Apellidos:García Sánchez
age:34


```

25

Peticiones asíncronas con async/await

- Elementos sintácticos para facilitar el trabajo con llamadas asíncronas (*callback's hell*)
- Introducidas en ES7 y soportadas por navegadores actuales
- **Definición de operadores:**
 - **await promesa**
 - Bloquea la ejecución de una función asíncrona hasta que se resuelve una promesa
 - Si la promesa se rechaza (*reject*) lanza una excepción
 - **async funcion**
 - Se ejecutan de forma asíncrona al ser llamadas
 - Devuelve un objeto *Promise*, por lo que al ser llamada no bloquea la ejecución
- **Ejemplo:**

```

window.onload = () => {
  document.getElementById( "btJson" )
    .onclick = () => {
      fetch( 'https://rest_service_url' )
        .then( response => response.json() )
        .then( datos => verDatos( datos ) )
        .catch( err => {
          console.log( "Error: ${err.message}" );
        });
    };
};

```

```

window.onload = () => {
  document.getElementById( "btJson" )
    .onclick = async () => {
      try {
        let resp = await
          fetch( 'https://rest service url' )
        let datos = await resp.json()
        verDatos( datos )
      } catch ( err ) {
        console.log( "Error: ${err.message}" );
      }
    };
};

```

Configuración de peticiones fetch

- Por defecto las peticiones fetch son de tipo GET
- En ocasiones se necesita modificar el tipo de petición:
 - Tipo de método: POST, PUT, DELETE...
 - Contenido del cuerpo (body), e.g. en peticiones POST o PUT
 - Encabezados específicos, e.g.
 - Tipo de contenido enviado, 'Content-type'
 - Tipo de contenido esperado: 'accept'
 - ...
- Configuración de la petición (objeto [Request](#) + [info](#))

fetch (url [, { request_config }])

- Objeto con elementos de petición: method, headers, body, cache, mode...
- Ejemplo de petición POST

```
fetch ( "api/clientes", {  
  method: 'POST',  
  body: JSON.stringify(nuevo_cliente),  
  headers: { 'Content-type': 'application/json',  
            'accept': 'application/json' }  
}).then ( response => response.json() ) // response.json() devuelve una promesa...  
.then ( datos => console.log (datos) ) //datos json recibidos y transformados  
.catch ( () => console.error( 'Error de conexión' ) )
```

27

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Ejemplo de petición JSON

```
<!-- file cliente.html... -->  
<head>  
  <script defer src="js/jquery.js"></script>  
  <script defer src="js/cliente.js"></script>  
</head>  
<body>
```

```
<h1>Localiza cliente por ID</h1>  
<input type="text" name="idbusca">  
<input type="button" value="Localiza">  
  
<div class="text-danger" id="errMsg"></div>
```

```
<div id="frm" hidden>  
  <form name="frmcliente" action="..." >  
    ID: <span id="id"></span>  
    Nombre: <input name="nombre">  
    DNI: <input name="dni">  
    Socio: <input name="socio" type="checkbox">  
    <input name="enviar" type="Submit" value="Guardar">  
  </form>  
</div>  
</body>  
28
```

Localiza cliente por ID



```
// file js/cliente.js  
$( () => {  
  $('[name=Localiza]').on('click', () => {  
    //Show selected cliente in edit form  
    let id=$('[name=idbusca']).val().trim() || 0;  
  
    let encontrado=false;  
    //Get cliente from server and fill form controls  
    fetch("api/clientes/"+id)  
      .then( response => {  
        if ( response.ok ) encontrado=true; //200  
        return response.json() } )  
      .then( datos => {  
        if ( encontrado ) {  
          let cliente=datos;  
          //fill form controls with cliente object data  
          $('[name=nombre]').val(cliente.nombre);  
          $('[name=dni]').val(cliente.dni);  
          $('[name=socio]').prop('checked', cliente.socio);  
        } else //e.g. status 400  
          $('#errMsg').html( 'el cliente no existe' );  
      })  
      .catch ( () => {  
        //Network error  
        console.error( 'Error de conexión' );  
        $('#errMsg').html( 'Error de conexión' );  
      })  
  });  
}); //End on-click event-handler  
}); //End DOM ready event handler
```

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Ejemplo app web CRUD AJAX con servicio JAX-RS

- Fuente: <http://github.com/jrbalsas/dawClubAJAX>
- Simple Page Application, SPA. Todo el código html se muestra en una página con actualizaciones AJAX. Fichero: [clientes.html](#)
 - Efecto de cuadro de diálogo modal de *Bootstrap*
 - Validación en cliente y servidor
- MVC Javascript (ES6) con clases controlador y DAOs alternativos usando fetch o \$ajax. Ficheros: [js/clientesController.js](#), [js/clientesDAOfetch/jquery.js](#)
- uso jQuery manipulación de VISTAS

Opciones Gestión CRUD Clientes AJAX/JAX-RS

Inicio

click en elemento para editar

ID	Nombre	DNI
1	Paco López	11111111A
2	María Jiménez	22222222B
3	Carlos García	33333333C

Edición de cliente

Cliente nº: 2

Nombre:

El nombre debe tener al menos 4 caracteres.

DNI:

Socio: ☐

29

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Ejemplo de controlador JS en lado del cliente

```
$( () => { //Configure events on page load
let clientesDAO= new ClientesDAOfetch( 'api/clientes' );
let viewModel = {
  clientes: [],
  cliente: {},
  errMsgs: []
};
//Create controller + Dependency Injection
let ctrl = new ClienteCtrl( viewModel , clientesDAO );
ctrl.init(); //Attach ev. handlers
});
class ClienteCtrl { //Clientes Controller
  constructor ( vm , clientesDAO ) {
    this.clientesDAO = clientesDAO; // DAO injection
    this.model=vm; //save view-model reference in controller
    this.config = { //place for common information
      wrapper: '#tbClientes', //<tbody> tag
      frmEdit: '#frmCliente', //... other config vars
    };
  }
  init() {
    //Attach view event-handlers
    $(this.config.frmEdit).submit( event => {
      event.preventDefault(); //Avoid default form submit
      this.frmSubmit();
    });
    //... Other view event-handlers
    this.loadClientes(); //Start Showing Clientes
  }
}
```

```
addCliente() {
  //Show empty form
}
editCliente(id) {
  //Show selected cliente in form
}
frmSubmit() {
  //Create or Update cliente on server
}
deleteCliente(id) {
}
validateCliente(cliente) {
  //Form Client-side validation
  //Shows validation errors next to form fields
}
frmEditShow() { //Shows Edit form
}
frmEditHide() { //Hides Edit form
}
frmEditUpdate() {
  //Fill form controls with this.model.cliente data
}
showServerErrors() {
  //Show Bean-Validation errors from server-side
}
loadClientes() {
  //Get clientes from server and update local model
}
showClientes() {
  //Fill table with clientes information
}
```

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Actualización de vista con contenidos JSON

```
loadClientes() {
    //Get clientes from server and update local model
    this.clientesDAO.buscaTodos()
        .then( clientes => {
            this.model.clientes = clientes;
            this.showClientes(); //force view update
        })
        .catch( errors => { //DAO or network errors
            this.model.errMsgs=errors;
            this.showServerErrors();
        });
}

showClientes() {
    //Fill table with clientes information
    let clientesRows = "";
    this.model.clientes.forEach( c => {
        //Place cliente id in user-defined row attribute for easy access
        //(see table click event)
        clientesRows += `<tr data-cliente-id="${c.id}">
            <td>${c.id}</td>
            <td>${c.nombre}</td>
            <td>${c.dni}</td>
            <td>${c.socio ? "Si" : "No"}</td>
        </tr>`;
    });
    $(this.config.wrapper).html(clientesRows);
}
```

```
<ul id="errMsgs" class="form-group">
    <!-- Server errors -->
</ul>

<table class="table table-striped table-hover">
    <thead>
        <tr><th>ID</th><th>Nombre</th><th>DNI</th>
        <th>Socio</th> </tr>
    </thead>
    <tbody id="tbClientes">
        <!-- Client list-->
    </tbody>
</table>

showServerErrors() {
    //Show BeanValidation errors from server-side
    let errorRows = "";
    this.model.errMsgs.forEach( (m)=> {
        errorRows += "<li class='text-danger'" + m.message
        + "</li>";
    });
    $(this.config.errMsgs).html(errorRows);
    this.model.errMsgs=[]; //Clean server errors
}
```

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Ejemplo de operación de Alta y Edición de Cliente

```
validateCliente(cliente) {
    //Form Client-side validation
    //Shows validation errors next to form fields
    let result = true;
    if (cliente.nombre.length < 4 ) {
        $('#errNombre').show();
        result = false;
    } else {
        $('#errNombre').hide();
    }
    //Further client-side input validations...
    //(omitted for checking server-side validation errors)
    return result;
} //End validateCliente

frmSubmit() {
    let cliente ={}

    //recover cliente data from Form
    cliente.id=      $('#id').text();
    cliente.nombre=  $('[name=nombre]').val();
    cliente.dni=     $('[name=dni]').val();
    cliente.socio=   $('[name=socio]').prop('checked');
```

```
//Form Client-side validation
if ( this.validateCliente(cliente) ) {

    //Create Edit (PUT) or Create (POST) DAO operation
    let operacionDAO=null;
    if (cliente.id > 0) {
        operacionDAO= this.clientesDAO.guarda(cliente);
    } else {
        operacionDAO=this.clientesDAO.crea(cliente);
    }

    operacionDAO
        .then( data => {
            this.frmEditHide();
            this.loadClientes(); //Refresh clientes list
        })
        .catch( errors => {
            this.model.errMsgs=errors;
            this.showServerErrors(); //Server-side validation errors
        });
} //End frmSubmit
```



Ejemplo de implementación DAO con peticiones

fetch

```
class ClientesDAOfetch {  
  
    constructor(apiUrl) { //pass REST api url, e.g. 'api/clientes'  
        this.srvUrl = apiUrl ;  
        this.respuestaValida=false; //status of last ajax request  
    }  
  
    buscaTodos() {  
        return fetch( this.srvUrl)  
            .then (response => this.comprobarRespuesta(response) )  
            .then (datos => this.devolverRespuesta(datos) );  
    }  
  
    busca(id = 0) {  
        return fetch(this.srvUrl + "/" + id)  
            .then (response => this.comprobarRespuesta(response) )  
            .then (datos => this.devolverRespuesta(datos) );  
    }  
  
    borra(id = 0) {  
        return fetch(this.srvUrl + "/" + id,{  
            method: 'DELETE'  
        })  
            .then (response => this.comprobarRespuesta(response) )  
            .then (datos => this.devolverRespuesta(datos) );  
    }  
}
```

33

```
crea(cliente) {  
  
    return fetch( this.srvUrl, {  
        method: 'POST',  
        body: JSON.stringify(cliente),  
        headers: {  
            'Content-type': 'application/json',  
            'accept': 'application/json'  
        }  
    })  
        .then (response => this.comprobarRespuesta(response))  
        .then (datos => this.devolverRespuesta(datos) );  
    }  
  
    guarda(cliente) { /*Igual que crea() con PUT*/  
        /** Check valid response and returns object data  
        comprobarRespuesta(response) {  
            this.respuestaValida=response.ok;  
            //TODO check network errors  
            return response.json();  
        }  
        /** Resolves or reject promise with response data*/  
        devolverRespuesta ( datos ) {  
            if ( ! this.respuestaValida ) {  
                //send validation errors  
                //Reject promise, forces catch response in Controller  
                return Promise.reject( datos );  
            }  
            return datos;  
        }  
    }; //End clientesDAOfetch
```

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Conclusiones

- Las aplicaciones web centradas en el cliente se basan en el intercambio de información estructurada con **servicios web** en el servidor
- **Jakarta RESTful web services, JAX-RS**, es la especificación estándar en Jakarta para el intercambio de datos HTTP en **formato JSON**
- El API de Javascript en el navegador soporta el intercambio asíncrono de datos estructurados con el servidor (AJAX) mediante la **función fetch** o el **objeto XMLHttpRequest**
- Las aplicaciones web también pueden estructurarse mediante **arquitecturas MVC en el cliente** con JS
- No obstante, implementar toda la lógica de control e interacción con la vista en JS es un **proceso costoso y propenso a errores**

34

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Bibliografía

- Building RESTful Web Services with Jakarta REST, en *Jakarta EE Tutorial*. Eclipse Foundation, 2024. Retrieved from <https://jakarta.ee/learn>
- D. R. Heffelfinger, *Jakarta EE Application Development: Build Enterprise Applications with Jakarta CDI, RESTful Web Services, JSON Binding, Persistence, and Security*. Packt Publishing, 2017. [[Recurso Electrónico](#)]
- Fetch API, in *Web technology for developers*. Mozilla Foundation, 2018. Retrieved April 2024 from https://developer.mozilla.org/es/docs/Web/API/Fetch_API
- Jakarta REST specification. Eclipse Foundation, 2024. Retrieved from <https://jakarta.ee/specifications/>

Lecturas Complementarias

- Rebecca Murphey. *Fundamentos de JQuery* (Leandro D'Onofrio). Uniwebsidad, 2006. Retrieved March 2025 from [[archive.org](#)]
- *Introducción a AJAX*. Uniwebsidad, 2006. Retrieved March 2025 from [[archive.org](#)]
- *AJAX*, in jQuery API Reference. jQuery Foundation. Retrieved April 2024 from <http://api.jquery.com/category/ajax/>

35

Apéndices

El objeto XMLHttpRequest en JS

● Métodos

- **open** ("GET", "/api/cliente/15") //Establecer conexión
- **setRequestHeader** ("Content-Type", "text/plain; charset=UTF-8")
- **send**(data) //Se envían los datos HTTP (o null si GET)
- **getResponseHeader** ("atributo"), **getAllResponseHeaders**(),
- **abort**() //terminar la conexión actual

● Eventos

- **.onreadystatechange**, se activa cuando hay novedades en la conexión

● Atributos

- **status**, **statusResponse** //e.g 200 "OK", 404 "Not Found"
- **responseText**, **responseXML**, //cuerpo de la respuesta HTTP
- **readyState**, //código de estado actual
 - 0 UNSET //conexión no establecida
 - 1 OPENED //se ha llamado a open()
 - 2 HEADERS_RECEIVED
 - 3 LOADING //Se está recibiendo el cuerpo de respuesta
 - 4 DONE //Respuesta completada

37

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Ejemplo petición asíncrona clásica

```
window.onload = function () {  
  2 document.getElementById( "btJson" ).onclick = function () {  
    var req = new XMLHttpRequest();  
    req.onreadystatechange = function () {  
      if ((req.readyState === req.DONE) && (req.status === 200)) {  
        3 verDatos( JSON.parse(req.responseText) );  
      }  
    };  
    req.open( 'GET', 'https://rest_service_url', true);  
    req.send(null);  
  };  
  function verDatos(obj) {  
    var dv1 = document.getElementById("id1");  
    var txt = "";  
    for (var prop in obj) {  
      txt += "<p><strong>${prop}</strong> ${obj[prop]}</p>";  
    }  
    dv1.innerHTML = txt;  
  };  
};
```

1

```
<body>  
  <div id="id1">Ejemplo JSON</div>  
  <button id="btJson" >Descarga datos</button>  
</body>
```

3 respuesta json de https://rest_service_url

```
{ "name": "Pepe", "Apellidos": "García Sánchez", "age": 34 }
```

4

```
name:Pepe  
Apellidos:García Sánchez  
age:34  
Press
```

38

Desarrollo de Aplicaciones Web. <http://tiny.cc/dawuja>. 2025



Ajax en jQuery

- Método **`$.ajax(url,{opciones}).then(manejador);`**

- Devuelve una promesa igual que *fetch*
- Los parámetros de configuración son específicos
- Existe una versión de jquery, *slim*, en la que se ha eliminado
- Ventaja: compatibilidad con navegadores no actualizados

- Ejemplos:

- Versión completa

```
$.ajax( "apis/clientes/"+id, //url en servidor
{
    type: "GET",
    data: null, //no hay datos en peticiones GET
    dataType: "json", //datos esperados
})
.then( function (cliente) {...} ); //manejador éxito
```

- Versión abreviada

```
$.getJSON( "api/clientes/"+id )
.then( function (cliente) {...} );
```

Con el tipo JSON, el objeto generado se pasa como parámetro al manejador (no es necesario usar `JSON.parse(json)`)

Comparación de peticiones asíncronas con `fetch()` y `$.ajax()`

Fetch API

```
window.onload = () => {
    document.getElementById( 'btJson' )
    .onclick = () => {

        fetch( 'https://rest_service_url' )
        .then ( resp => resp.json() )
        .then ( obj => verDatos(obj) )
        .catch( () => console.log("Error conexión") );

    };
};
```

```
function verDatos( obj ) {
    var dv1 = document.getElementById("id1");
    var txt = "";
    for (var prop in obj) {
        txt += "<p><strong>${prop}</strong> ${obj[prop]}<p>";
    }
    dv1.innerHTML = txt;
};
```

JQuery AJAX Api

```
<script src="//code.jquery.com/jquery-3.3.1.min.js"></script>
```

```
window.onload = () => {
    document.getElementById("btJson")
    .onclick = () => {
```

```
$.ajax( 'https://rest_service_url' )
    .then ( obj => verDatos(obj) )
    .catch ( error =>
        console.log(error)
    );
};
```